Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică
Departamentul Ingineria Software și Automatică
Program: Software Engineering

Operating Systems

Laboratory Work No. 2

Meet the OS You Will Build

Track B: Building and Exploring xv6

Done by:
st. gr. FAF-241
Serghei Barladean

Checked by:
Patricia Reitman

Chișinău – 2026

Contents

1. Purpose of the Work..... 3
2. Building and Booting xv6..... 3
3. Trying the xv6 Shell..... 3
3.1 What I Ran..... 3
3.2 Answers to the Observe Questions..... 4
4. Reading the Source..... 4
4.1 System Calls in cat.c..... 5
4.2 Where the Kernel Handles read and write..... 5
4.3 kernel/ Versus user/..... 5
5. Writing My Own Program: sleep..... 6
5.1 Testing in xv6..... 6
6. Conclusions..... 6

1. Purpose of the Work

I did this laboratory work to get acquainted with xv6, a small teaching operating system written in C that follows the design of early Unix. The plan was to build it from source, boot it in QEMU, try its shell, read a few of its files and add one program of my own. While doing this I wanted to see where ordinary programs end and the kernel begins.

2. Building and Booting xv6

Everything was done in the Ubuntu 24.04 virtual machine (Multipass, aarch64) that I use for all the labs, because the macOS host cannot build the RISC-V version directly. I cloned the mit-pdos/xv6-riscv repository into ~/xv6-riscv and installed the toolchain with one command:

```
sudo apt install -y git build-essential gdb-multiarch \
    qemu-system-misc gcc-riscv64-linux-gnu \
    binutils-riscv64-linux-gnu
```

The command make qemu compiles the kernel and every program from user/, packs them into a filesystem image and starts the QEMU emulator. The boot messages ended with the shell prompt $, so the kernel and the first process init were working (Figure 1). To leave QEMU I pressed Ctrl-A and then X.

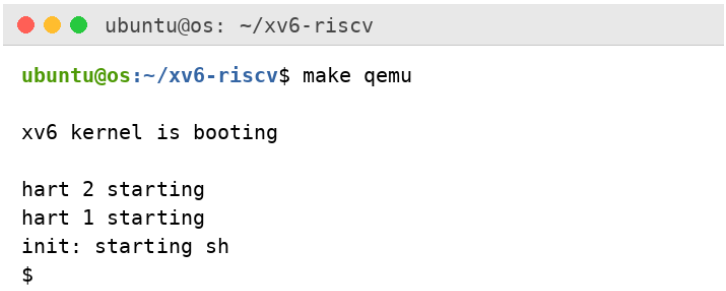


Figure 1. Building and booting xv6 with make qemu

3. Trying the xv6 Shell

3.1. What I Ran

At the xv6 prompt I ran four commands: ls, echo hello xv6, ls | grep c and wc README. The ls command printed 24 entries (Figure 2). Each line has the name, the file type (1 is a directory, 2 a file, 3 a device), the inode number and the size in bytes. The echo command simply printed hello xv6.

The pipeline ls | grep c kept five of those lines: cat, echo, wc, sync and console, the names that contain the letter c. The command wc README answered 48 336 2441, that is 48 lines, 336 words and 2441 bytes, and the byte count matches the size of README in the ls listing (Figure 3).

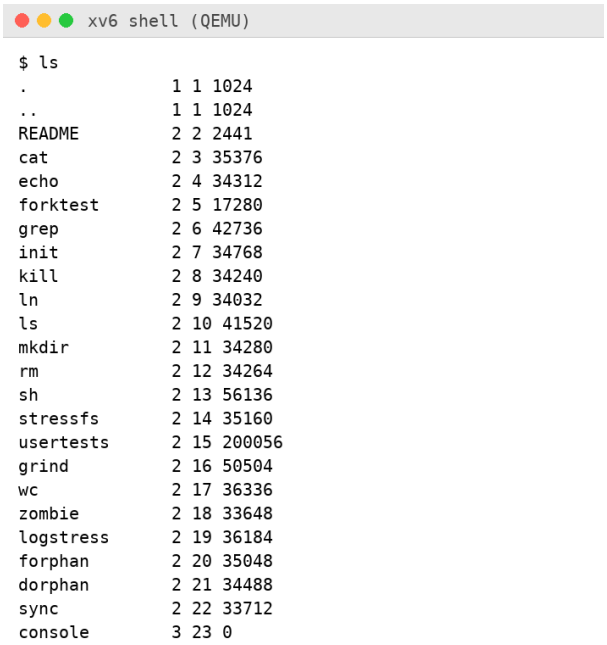


Figure 2. Programs available in the xv6 filesystem (ls)

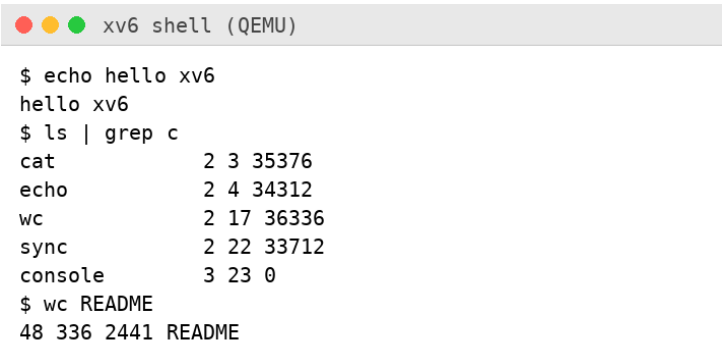


Figure 3. echo, a pipe with grep, and wc in the xv6 shell

3.2. Answers to the Observe Questions

1. Besides the programs I know from Linux, such as ls, grep, mkdir, rm, kill and ln, the listing contains cat, echo and wc, which I used directly.
2. A pipe needs two things from the operating system. The first is several processes: for ls | grep c the shell creates one child process per command with fork and turns each child into the right program with exec. The second is a way for these processes to talk to each other: the pipe system call creates a buffer inside the kernel with a write end and a read end, and the shell connects the output of ls to the write end and the input of grep to the read end.
3. The xv6 shell is a much smaller relative of Bash. The basic ideas are the same (descriptors 0, 1 and 2, pipes, programs started with fork and exec), but it is far less comfortable to use.

4. Reading the Source

I read user/ls.c, user/cat.c and user/user.h, and searched the kernel with grep -n "sys_read\| sys_write" kernel/sysfile.c.

4.1. System Calls in cat.c

The program cat.c uses five system calls: open gets a descriptor for a file, read copies up to n bytes from a descriptor into a buffer, write sends n bytes from the buffer to a descriptor (here descriptor 1, the screen), close releases the descriptor and exit ends the process with a status code. The core of the program is a loop that reads a block and writes it out until read returns zero:

```
while((n = read(fd, buf, sizeof(buf))) > 0)
  write(1, buf, n);
```

4.2. Where the Kernel Handles read and write

In user.h the functions read() and write() are only declared; the real work happens inside the kernel. The search showed that sys_read begins at line 69 and sys_write at line 83 of kernel/sysfile.c. So a call to read() in cat makes the processor switch into the kernel, which runs sys_read on behalf of the program (Figure 4).

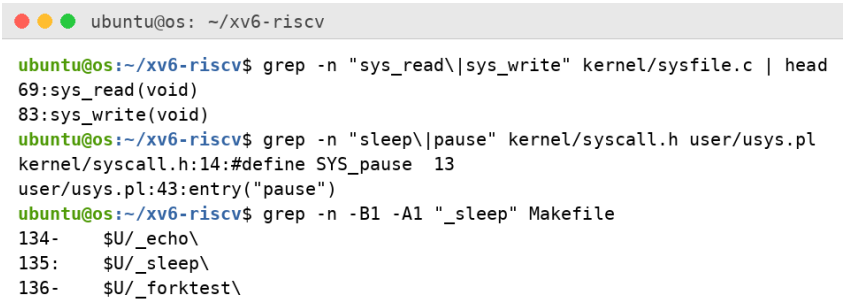


Figure 4. Locating sys_read and sys_write, the pause system call, and the Makefile entry

4.3. kernel/ Versus user/

The folder kernel/ holds the code that runs with full privileges: the scheduler, memory management, the file system and the system-call handlers. The folder user/ holds ordinary programs such as ls, cat, the shell and my own sleep. A user program cannot touch the hardware or kernel memory directly, the only way in is a system call. This is why a faulty user program can crash itself but not the whole system.

5. Writing My Own Program: sleep

The task was a user program sleep that pauses for a given number of ticks. The example in the handout calls sleep(), but in the xv6 version I cloned this system call is named pause(): grep over user/user.h finds only pause, and kernel/syscall.h defines SYS_pause. My user/sleep.c therefore looks like this:

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1]));
  exit(0);
}
```

If there is not exactly one argument the program prints a usage message to descriptor 2 (the error output) and exits with status 1. Otherwise atoi turns the argument into a number and pause waits for that many clock ticks.

To get the program built I added the line $U/_sleep\ to the UPROGS list in the Makefile, with a tab at the start. I added it before the source file existed, and make stopped with the message that there is no rule to make target user/_sleep. It continued only after user/sleep.c was created. This showed me that every entry of UPROGS needs a matching .c file, and that the Makefile is what puts the finished program into the filesystem image.

5.1. Testing in xv6

My first test was wrong: typing sleep 10 in the Ubuntu terminal runs the Linux sleep and says nothing about my program. The program has to be tested at the xv6 prompt after make qemu. There, sleep 10 waited for a moment and returned to the prompt, and sleep without an argument printed usage: sleep <ticks> (Figure 5).

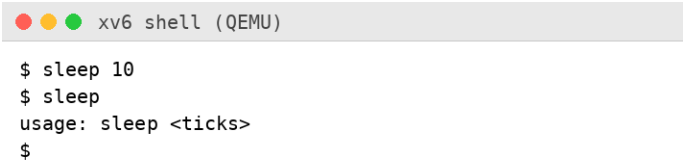


Figure 5. Testing the sleep program inside xv6

6. Conclusions

xv6 builds with a single command and boots in QEMU within seconds, which makes it a convenient place to look at a Unix system from the inside. Its shell already gives pipes, and they rest on three system calls: fork, exec and pipe.

Reading cat.c showed me how little an ordinary program does by itself: it opens a file, reads and writes blocks and exits, and everything privileged happens behind system calls whose handlers live in kernel/sysfile.c.

Writing sleep taught me two practical things: the system call differs between xv6 versions (pause instead of sleep), and a new program only exists after it is listed in UPROGS and the source file is present. In the end the program compiled, ran in xv6 and reported the missing argument correctly.